SPLITEASE: COMPREHENSIVE PROJECT REPORT

1. PROJECT OVERVIEW

Project Name: SplitEasy (Repository: crosshoc)
Version: 0.1.0
Type: Full-stack web application for expense splitting and group financial management
Current Phase: Phase 5 Complete → Phase 6 (Mobile-Native UX) In Progress

Purpose:
SplitEasy is a modern expense-splitting application that enables users to:

- Create and manage shared expense groups
- Split expenses fairly among group members
- Track who owes whom
- Generate settlement instructions
- Share groups via invite links
- Support guest (unauthenticated) users

Key Differentiators:

- Guest user support with email-based activation
- Magic link invitations for frictionless onboarding
- Real-time balance calculations
- CSV export capabilities
- Rolling session management with JWT tokens

2. TECH STACK

Frontend

- **Framework:** Next.js 16.1.6 (React 19.2.3)
- **Language:** TypeScript 5
- **Styling:** Tailwind CSS 4 (@tailwindcss/postcss)
- **UI Component Library:** shadcn/ui (via Base UI + custom components)
- **Icons:** Lucide React v0.577.0
- **Animations:** Framer Motion v12.38.0
- **Toast Notifications:** Sonner v2.0.7
- **QR Code:** qrcode.react v4.2.0
- **Utility Libraries:**
  - `clsx` - Conditional CSS class merging
  - `tailwind-merge` - Intelligent Tailwind class merging
  - `class-variance-authority` - Component variant patterns
  - `tw-animate-css` - Animation utilities

Backend

- **Runtime:** Next.js App Router (Edge Runtime compatible)
- **Database:** MongoDB v9.5.0 (via Mongoose)
- **Authentication:**
  - `jose` v6.2.2 - JWT signing/verification (Edge Runtime)
  - `jsonwebtoken` v9.0.3 - Token generation/validation
  - `bcryptjs` v3.0.3 - Password hashing
- **Session Management:** HTTP-only cookies with rolling refresh
- **Image Storage:** Cloudinary (via next-cloudinary v6.17.5)

Developer Tools

- **Linting:** ESLint 9
- **Package Manager:** pnpm (workspace enabled)
- **Node Type Definitions:** @types/node v20
- **Deployment:** Vercel (vercel.json config present)

3. ARCHITECTURE

A. Project Structure

```
spliteasy/
├── app/                          # Next.js App Router
│   ├── (auth)/                   # Auth routes group
│   │   ├── login/
│   │   └── register/
│   ├── (dashboard)/              # Protected dashboard routes
│   │   ├── layout.tsx            # Dashboard shell
│   │   ├── page.tsx              # Main dashboard
│   │   ├── dashboard/            # Dashboard view
│   │   ├── groups/               # Group management
│   │   ├── expenses/             # Expense feed
│   │   ├── settlements/          # Settlement tracking
│   │   ├── settings/             # User settings
│   │   ├── help/                 # Help/documentation
│   │   └── feedback/             # Feedback form
│   ├── api/                      # REST API routes
│   │   ├── auth/                 # Authentication endpoints
│   │   ├── balances/             # Balance calculation
│   │   ├── expenses/             # Expense CRUD
│   │   ├── groups/               # Group management
│   │   ├── guest/                # Guest user endpoints
│   │   └── user/                 # User profile
│   ├── invite/                   # Public invite pages
│   ├── join/                     # Magic link join routes
│   ├── layout.tsx                # Root layout
│   ├── page.tsx                  # Landing page
│   └── globals.css               # Global styles
├── components/                   # React components
│   ├── ui/                       # Reusable UI primitives
│   ├── app-sidebar.tsx           # Main sidebar navigation
│   ├── dashboard-shell.tsx       # Dashboard layout wrapper
```

```
│   ├── ExpenseCard.tsx           # Expense display
│   ├── SettlementCard.tsx        # Settlement display
│   └── [other components]
├── lib/                          # Shared utilities
│   ├── models/                   # Mongoose models
│   ├── auth.ts                   # Authentication logic
│   ├── db.ts                     # Database connection
│   ├── balance-*.ts              # Balance calculations
│   └── [utilities]
├── hooks/                        # Custom React hooks
│   └── use-mobile.ts             # Mobile detection
├── middleware.ts                 # Request middleware
├── package.json                  # Dependencies
├── tsconfig.json                 # TypeScript config
├── next.config.ts                # Next.js config
├── tailwind.config.ts            # Tailwind config
├── postcss.config.mjs            # PostCSS config
└── vercel.json                   # Vercel deployment config
```

B. Frontend Architecture

Layout Hierarchy (Current):

```
Root Layout (app/layout.tsx)
  ├── Auth Routes (app/(auth)/login, register)
  ├── Dashboard Routes (app/(dashboard)/layout.tsx)
  │   └── Dashboard Shell (components/dashboard-shell.tsx)
  │       ├── Sidebar (components/app-sidebar.tsx)
  │       ├── Feed (expenses, groups, settlements)
  │       └── Summary (balance info, members)
  └── Public Routes (landing, invite, join)
```

Component Organization:

- **Primitives** (ui) - Reusable, unstyled components (button, card, input, etc.)
- **Composed** (components) - Feature-specific components (ExpenseCard, SettlementCard, etc.)
- **Pages** (app) - Route-specific layouts and content

State Management:

- React hooks (useState, useContext)
- Server-side data fetching (getServerSideProps-equivalent via API routes)
- Client-side mutations (fetch API from client components)
- No Redux/Zustand detected - lightweight approach

Styling Strategy:

- **Tailwind CSS v4** - Utility-first CSS framework
- **CSS-in-JS:** Minimal (no styled-components detected)
- **Theme:** Light/Dark mode support via CSS variables (Tailwind)

- **Breakpoints:** Default Tailwind (`sm: 640px`, `md: 768px`, `lg: 1024px`, `xl: 1280px`)

C. Backend Architecture

API Structure:

- **Routing:** Next.js App Router API routes (`app/api/[route]/route.ts`)
- **Pattern:** RESTful endpoints with specific methods (GET, POST, PUT, DELETE)
- **Middleware:** Custom middleware for auth validation

Data Models (MongoDB):

1. **User** - User accounts with credentials
2. **Group** - Expense groups
3. **Expense** - Individual expenses with splits
4. **Settlement** - Settlement instructions
5. **GuestSession** - Guest user sessions
6. **InviteToken** - Time-limited invite links
7. **GuestSettlement** - Settlements for guest users

Authentication Flow:

```
User Login/Register → JWT Token Created → Stored in HTTP-Only Cookie
                                    ↓
Middleware Validates Token on Each Request
                                    ↓
Rolling Session: Token Expires in 30 days
If Token < 24 hours remaining → Auto-refresh with new 30-day expiration
```

Key Middleware Logic (middleware.ts):

- Checks token validity using `jose` (Edge Runtime compatible)
- Handles rolling session refresh
- Redirects unauthenticated users from protected routes
- Public routes: `/`, `/login`, `/register`, `/invite/*`, `/join/*`

4. CORE FEATURES

A. Expense Management

Files: expenses, ExpenseCard.tsx

Features:

- Create new expenses with amount, category, date, description
- Assign payer and split details
- Edit/delete existing expenses
- Categorization (food, transport, entertainment, utilities, other)
- Expense feed with pagination/filtering

API Endpoints:

```
GET  /api/expenses           - List expenses
POST /api/expenses           - Create expense
GET  /api/expenses/[id]      - Get single expense
PUT  /api/expenses/[id]      - Update expense
DEL  /api/expenses/[id]      - Delete expense
```

B. Group Management

Files: groups, [groupId]

Features:

- Create/edit/delete groups
- Add members to groups
- Generate invite tokens with magic links
- Refresh expired invites
- Export group expenses as CSV
- Group settings and member management

API Endpoints:

```
GET  /api/groups             - List user's groups
POST /api/groups             - Create group
GET  /api/groups/[id]        - Get group details
PUT  /api/groups/[id]        - Update group
DEL  /api/groups/[id]        - Delete group
POST /api/groups/[id]/settle - Settle up
POST /api/groups/[id]/invite - Generate invite
POST /api/groups/[id]/export-csv - Export expenses
```

C. Balance Calculation & Settlement

Files: balance.ts, balance-server.ts, balance-types.ts

Features:

- Real-time balance calculation (who owes whom)
- Optimal settlement suggestions (minimizes transactions)
- Transaction history
- Support for partial settlements
- Balance summary by user

Algorithm:

- Tracks cumulative balance per user
- Generates settlement matrix

- Suggests most efficient payment plan

D. Guest User Management

Files: guest, invite

Features:

- Create guest sessions without signup
- Email-based guest invitations
- Guest expense viewing
- Guest settlement information
- Activate guests to full users (via email)

Flow:

```
Guest Receives Invite Link → View Expenses (No Auth)
                                ↓
            Click "Claim Account" → Email Verification
                                ↓
         Create Password & Account → Full Access
```

E. Authentication & Authorization

Files: auth.ts, auth, middleware.ts

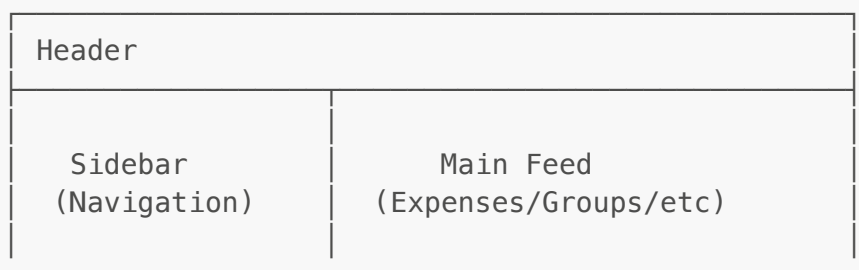
Features:

- User registration with email/password
- Login with JWT token
- Logout (cookie deletion)
- Session validation
- Rolling token refresh
- Guest session management
- Group membership verification

5. UI/UX DESIGN

A. Current Layout Structure

Desktop (lg screens, 1024px+):



```
│                    │  ┌──────────────────────────┐
│                    │  │ Summary/Balance Panel    │
│                    │  │ Members List             │
│                    │  └──────────────────────────┘
│                    │                              │
└────────────────────┴──────────────────────────────┘
```

Current Mobile Behavior:

- Sidebar likely remains visible (not optimized)
- Content squished horizontally
- Touch targets may be too small
- Sticky actions not implemented
- Modal dialogs may have poor mobile UX

B. Navigation Structure

Primary Navigation (Sidebar):

- Dashboard
- Groups (with group list)
- Expenses
- Settlements
- Settings
- Help
- Feedback

Secondary Navigation:

- User profile / logout
- App branding

C. Key Components & Their Roles

Component	Purpose	Location
`app-sidebar.tsx`	Main navigation menu	Desktop & mobile (drawer)
`dashboard-shell.tsx`	Layout wrapper for dashboard	layout.tsx
`ExpenseCard.tsx`	Display individual expense	Feed, group detail
`SettlementCard.tsx`	Display settlement instruction	Settlements view
`share-group-dialog.tsx`	Share invite link	Group detail
`settle-up-button.tsx`	Trigger settlement flow	Group view
`ExportCsvButton.tsx`	Download expenses	Group view

D. Visual Style

Color Scheme:

- Primary: Emerald (emerald-600 for actions)
- Background: Light/Dark mode support
- Text: Gray gradations for hierarchy
- Accent: Emerald for CTAs

Typography:

- Body text: 14-16px (likely via Tailwind defaults)
- Headings: Multiple font weights (semibold, bold)
- Monospace: For financial amounts (if applicable)

Spacing:

- Gaps: `gap-2`, `gap-3`, `gap-4` (8px, 12px, 16px units)
- Padding: `p-3`, `p-4` on cards
- Margins: Standard Tailwind spacing

6. MOBILE RESPONSIVENESS

A. Current State

What's Implemented:

- Tailwind CSS responsive classes (sm, md, lg, xl)
- Media query support in CSS
- Sidebar component (but placement unclear for mobile)
- Some dialog components for modals

What's Missing (Phase 6 Target):

- ❌ Mobile-first navigation (no sheet/drawer on mobile)
- ❌ Thumb-first layout optimization
- ❌ Sticky header with hamburger menu
- ❌ Bottom sticky action bar
- ❌ 44x44px minimum touch targets
- ❌ Full-screen mobile input forms
- ❌ Optimized card layouts for small screens
- ❌ Dark mode contrast fixes
- ❌ Performance optimization for mobile

B. Breakpoint Strategy

Current Tailwind breakpoints (default):

```
sm:  640px   - Small phones
md:  768px   - Tablets
lg:  1024px  - Desktops (likely primary breakpoint)
xl:  1280px  - Large screens
```

Observation: No custom breakpoints detected; using Tailwind defaults.

7. STRENGTHS

Architecture

✅ **Modern Tech Stack** - Next.js 16, React 19, TypeScript 5
✅ **Edge Runtime Ready** - Middleware uses `jose` for Edge compatibility
✅ **Type Safety** - Full TypeScript coverage
✅ **Scalable Structure** - App Router with clear separation of concerns

Features

✅ **Guest Support** - Unique feature for frictionless onboarding
✅ **Smart Settlements** - Optimal transaction calculation
✅ **Magic Links** - No password required for joining
✅ **Data Export** - CSV download capability
✅ **Session Management** - Rolling token refresh for better UX

Security

✅ **JWT Token Auth** - Stateless, scalable authentication
✅ **HTTP-Only Cookies** - Protected against XSS
✅ **Bcrypt Hashing** - Secure password storage
✅ **Rolling Sessions** - Automatic token refresh without logout friction

Developer Experience

✅ **Clear Component Structure** - UI primitives + composed components
✅ **Utility-First CSS** - Tailwind for rapid development
✅ **Type Definitions** - All models fully typed
✅ **API Route Organization** - Clear RESTful patterns

8. LIMITATIONS & ISSUES

Mobile UX

❌ **Not Mobile-Native** - Sidebar remains visible on small screens
❌ **No Thumb-Friendly Layout** - Content not optimized for one-handed use
❌ **Missing Bottom Actions** - No sticky action buttons for mobile
❌ **Likely Layout Shifts** - Potential CLS issues during interactions
❌ **Input UX** - Modals may not be full-screen on mobile

Accessibility

❌ **Unclear ARIA Labels** - Limited accessibility attributes (need audit)
❌ **Touch Targets** - Unlikely to meet 44x44px minimum (no explicit enforcement)
❌ **Dark Mode Contrast** - May have low-contrast text issues

Performance

❌ **No Detected Caching** - No SWR, React Query, or service worker setup
❌ **Potential N+1 Queries** - Need to verify API optimization
❌ **Image Optimization** - No explicit Next.js Image component usage detected

Data Handling

❌ **No Real-Time Updates** - No WebSocket/polling detected
❌ **No Optimistic Updates** - Client-side mutations likely require full refetch
❌ **No Offline Support** - No service worker or offline-first capability

Scalability Concerns

❌ **MongoDB Scalability** - No sharding/clustering strategy visible
❌ **No Caching Layer** - No Redis/Memcached detected
❌ **API Rate Limiting** - Not visible in route handlers

9. RECOMMENDATIONS & FUTURE IMPROVEMENTS

PHASE 6: Mobile-Native UX (IMMEDIATE)

Priority 1 - Navigation & Layout:

1. Replace sidebar with Sheet/Drawer on screens < 1024px
2. Add sticky top header with:
    - Hamburger menu (left)
    - App title (center)
    - "+ Add Expense" button (right)
3. Reorder feed for mobile (Feed → Summary → Members)
4. Add sticky bottom action bar (Add Expense + Settle Up)

Priority 2 - Touch Optimization:

1. Ensure all interactive elements are 44x44px minimum
2. Increase vertical spacing in cards and dropdowns
3. Replace hover interactions with visible icons/tap-to-expand
4. Add visual feedback (scale, opacity changes) on tap

Priority 3 - Input Experience:

1. Convert modals to full-screen mobile drawers
2. Add `text-base` to inputs to prevent iOS zoom
3. Keep form actions (Save/Cancel) sticky at bottom
4. Improve date/category pickers for touch

Priority 4 - Polish:

1. Fix dark mode contrast (especially guest names, balances)
2. Test animations for lag on mobile (simplify if needed)
3. Prevent layout shifts during card expansion (CSS containment)

4. High contrast for balance amounts (use emerald-400 in dark mode)

PHASE 7: Performance & Advanced Features

Data Fetching:

1. Implement SWR or React Query for:
    - Automatic revalidation
    - Optimistic updates
    - Mutation handling
2. Add API response caching
3. Implement React Suspense for better loading states

Real-Time Updates:

1. Add WebSocket connection for live balance updates
2. Implement server-sent events (SSE) for settlements
3. Real-time notifications for group changes

Offline Support:

1. Service worker for offline-first capability
2. IndexedDB for local storage of expenses
3. Sync queue for offline mutations

Mobile Features:

1. Push notifications (PWA)
2. App installation prompt
3. Native share functionality

PHASE 8: Scalability & Backend

Database Optimization:

1. Add database indexing strategy
2. Implement connection pooling
3. Optimize balance calculation queries

API Enhancement:

1. Add GraphQL as alternative to REST
2. Implement API versioning
3. Add comprehensive error handling and logging
4. Rate limiting per user

Monitoring & Analytics:

1. Error tracking (Sentry)
2. Performance monitoring (Vercel Analytics)
3. User behavior analytics
4. A/B testing framework

PHASE 9: Advanced Features

Group Features:

1. Recurring expenses
2. Budget tracking and alerts
3. Group spending analytics/charts
4. Admin roles with permissions

Payment Integration:

1. Stripe integration for direct payments
2. PayPal/Square integration
3. Bank transfer instructions
4. Crypto payment option

Social Features:

1. User profiles
2. Friend management
3. Group comments/chat
4. Activity feed/notifications

10. CODE QUALITY ASSESSMENT

Strengths

- Clear file organization and naming
- Consistent component structure
- Type safety with TypeScript
- Proper use of async/await
- Good separation of concerns

Areas for Improvement

- Add comprehensive error handling
- Implement consistent loading states
- Add form validation library (Zod, Yup)
- Create shared hooks for common patterns
- Add integration tests
- Document complex algorithms (balance calc)

11. SECURITY AUDIT

✅ Secure Practices

- HTTP-only cookies for tokens
- CORS handling (need to verify)
- Input validation (need to verify)
- Password hashing with bcryptjs

- JWT signature verification
- Protected routes with middleware

⚠ Areas to Verify

- SQL injection protection (MongoDB injection)
- CSRF token handling
- Rate limiting on auth endpoints
- File upload validation (if applicable)
- Sensitive data logging

SUMMARY SCORECARD

Category	Score	Status
Code Quality	8/10	Good - Type safe, organized
Architecture	8/10	Strong - Clear structure
Features	8/10	Rich - Guest support unique
Security	7/10	Good - Need audit
Mobile UX	3/10	Poor - Not optimized
Performance	6/10	Adequate - No caching
Accessibility	5/10	Needs work - A11y audit needed
Documentation	4/10	Minimal - Add more docs
Testing	2/10	Missing - Add test coverage
Deployment	9/10	Excellent - Vercel ready

Overall: 6.0/10 - **Solid foundation, needs mobile polish & performance optimization**

RECOMMENDED NEXT STEPS

1. ✅ **IMMEDIATELY** - Implement Phase 6 (Mobile-Native UX)
2. ➡️ **Week 2** - Add data fetching library (SWR)
3. ➡️ **Week 3** - Dark mode contrast fixes + accessibility audit
4. ➡️ **Week 4** - Performance optimization (images, fonts, bundle)
5. ➡️ **Month 2** - Real-time features + offline support
6. ➡️ **Month 3** - Advanced features + payment integration
